24CSA212 – Full Stack Frameworks

BASICS OF HTML, CSS, AND JAVASCRIPT -INTRODUCTION TO MONGODB

Dynamic vs. Static Web Sites

Web pages can be either static or dynamic. "Static" means unchanged or constant, while "dynamic" means changing or lively. Therefore, static Web pages contain the same prebuilt content each time the page is loaded, while the content of dynamic Web pages can be generated on-the-fly.

- *What is Web content? Content is everything that can appear on a Web page:*

  text, graphics, form fields, hyperlinks to other pages, navigation buttons, menus, etc.

Advantages of static websites

- Quick to develop
- Cheap to develop
- Cheap to host

Disadvantages of static websites

- Requires web development expertise to update site
- Site not as useful for the user
- Content can get inactive

Advantages of dynamic websites

- Much more practical website
- Much easier to update
- New content brings people back to the site and helps in the search engines
- Can work as a system to allow staff or users to collaborate
- **Disadvantages of dynamic websites**
- Slower / more expensive to develop
- Hosting costs a little more

- Standard HTML pages are static Web pages. Each time an HTML page is loaded, it looks the same. The only way the content of an HTML page will change is if the Web developer updates and publishes the file

- Other types of Web pages, such as PHP, ASP, and JSP pages are dynamic Web pages. These pages contain "server-side" code, which allows the server to generate unique content each time the page is loaded.

- database information, which enables the page's content to be generated from information stored in the database. Websites that generate Web pages from database information are often called database-driven websites.

- You can often tell if a page is static or dynamic simply by looking at the page's file extension in the URL, located in the address field of the Web browser. If it is ".htm" or ".html," the page is probably static.

- If the extension is ".php," ".asp," or ".jsp," the page is most likely dynamic. While not all dynamic Web pages contain dynamic content, most have at least some content that is generated on-the-fly.

Planning your site online

In order to be able to upload a web Page to the Internet, it's necessary to hire a web hosting company to make your web site available to others on the internet 24 hours a day.

The price to have your own space on their servers will depend on the company, the space on disk you need,

For these reasons, it is not recommended to use a free hosting for a bussiness Page, although it is acceptable for a personal one.

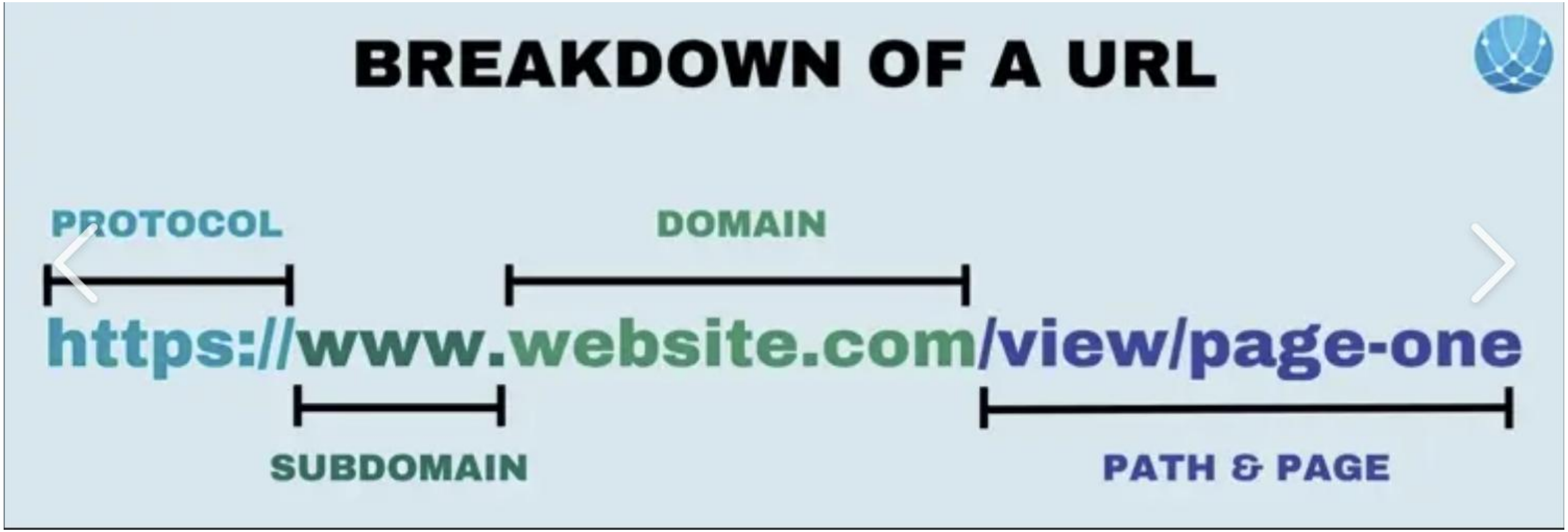
- There are companies that offer this service for free, but with certain limitations: just a few disk space, slowness and a name for our Page preceded by their own etc...

- It's also necessary to consider that these companies must generate income in some way it is the reason why they are dedicated to sell advertising spaces within your Pages, ads that we'll not be able to avoid including in our web Pages.

Planning your site online

- When we are going to hire a web hosting service it is necessary to also contract a **domain**, a task which usually is in charge of the hosting company. To register a domain consists of registering a name for our Page. This name cannot be repeated in Internet, has to be unique, as happens with the names of the companies. It's possible to register the same name with different extensions, like for example, .net, .org, .biz or .com.

Universal Resource Location (URL)



Domain name

co
com
ac
edu
gov
org

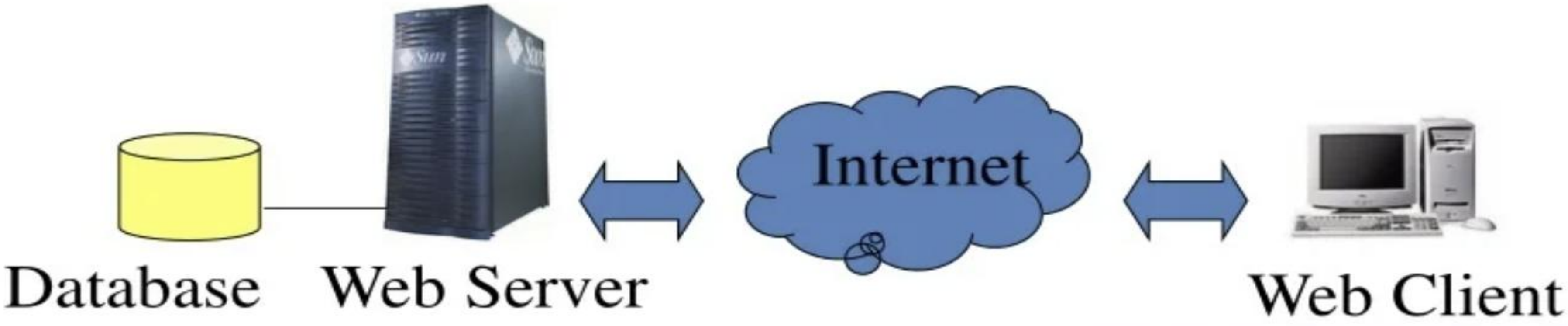
www.gtslearning. **the domain name of the server.**

.co identifies the website as a commercial organisation. It is convention that commercial sites use **co or com**

ac or edu (for academic or education)

gov and org use for governmental or non-profit organisations

.co.uk indicates that the site belongs to a commercial organisation registered in the UK



Server-side Programming

Skills that are often required:

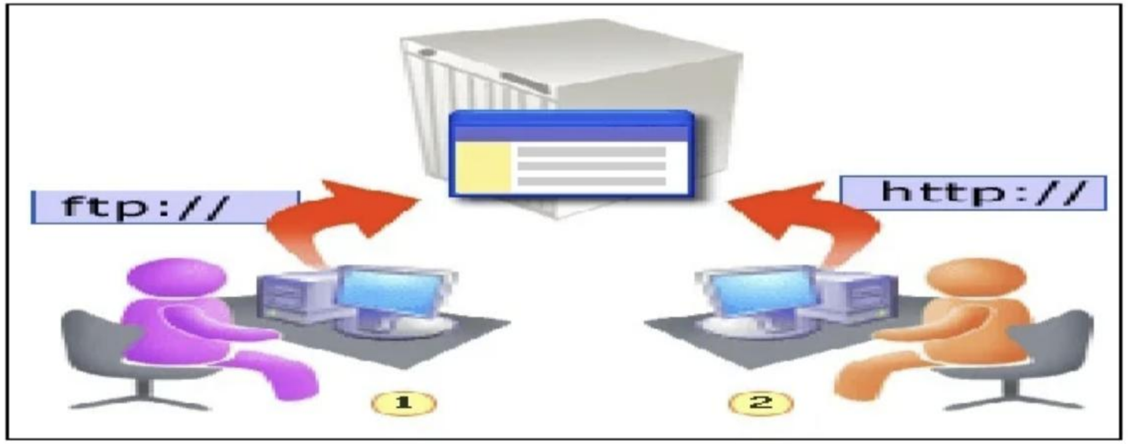
- CGI
- PHP
- ASP
- Perl
- Java Servlet, …

Client-side Programming

Skills that are often required:

- XHTML
- Javascript
- Java
- Dreamweaver
- Flash
- SMIL, XML …

Publish Your Pages



1) This designer is publishing a Web site through the "back door" using FTP.

2) After the publishing process is complete, the site is available to the public. People visiting your site use the "front door" via a Web browser.

3) With this in mind, think of the FTP address as the "back door" that you deliver your Web pages to, and

   the HTTP address as the "front door" that people use to see your pages

Introduction to web technologies:

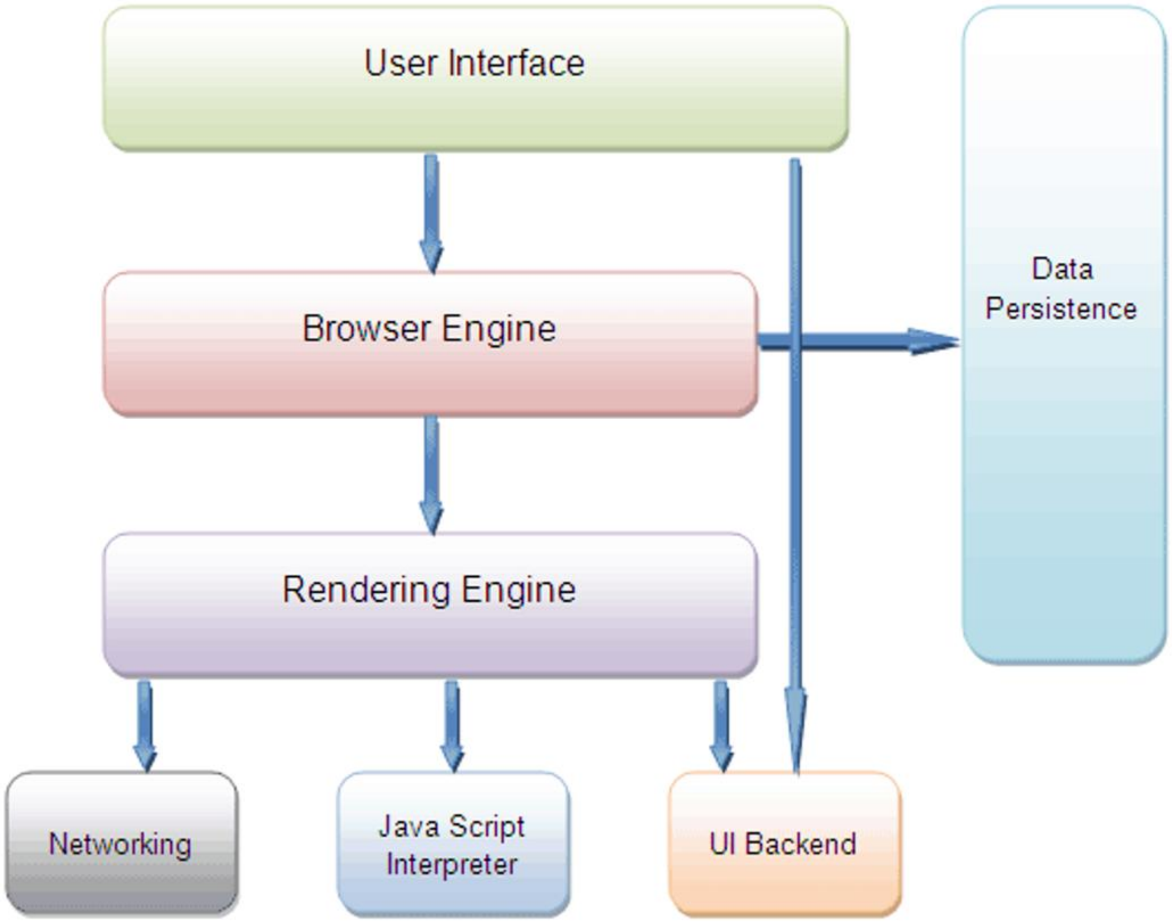
- HTML to create the document structure and content
- CSS to control is visual aspect
- Javascript for interactivity

Inside a Browser

Browsers have very differentiate parts.

We are interested in two of them:

- the Rendering Engine (in charge of transforming our HTML+CSS in a visual image).
- The Javascript Interpreter (also known as VM), in charge of executing the Javascript code.

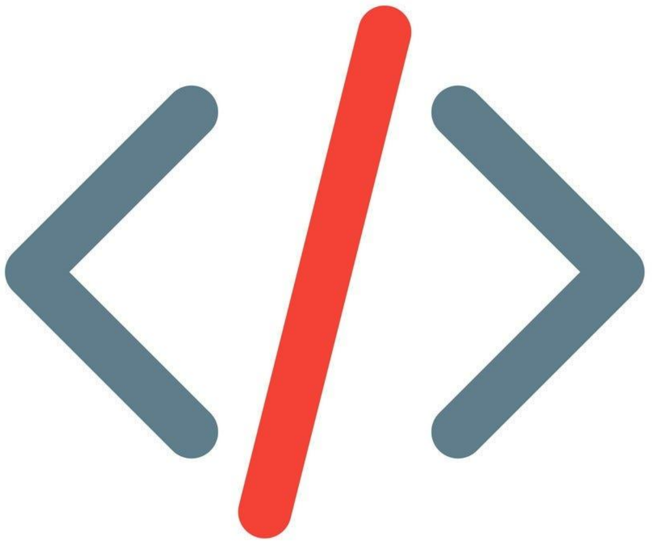


Markup language

There are many markup languages that add special tags into the text that the renderer wont show but use to know how to display the text.

In HTML this tags use the next notation:

```
My name is <b>XYZ</b>
```



HTML

HTML means **Hyper Text Markup Language**.

The HTML allow us to define the structure of a document or a website.

HTML is **NOT** a programming language, it’s a markup language, which means its purpose is to give structure to the content of the website, not to define an algorithm.

It is a series of nested tags (it is a subset of XML) that contain all the website information (like texts, images and videos).

Example:-

```
<title>This is a title</title>
```

The HTML defines the page structure. A website can have several HTMLs to different pages.

```
<html>
    <head>
    </head>
    <body>
        <div>

    <p>Hi</p>
        </div>
    </body>
</html>
```

HTML: Basic rules

Some rules about HTML:

- It uses XML syntax (tags with attributes, can contain other tags).

```
<tag_name attribute="value"> content </tag_name>
```

- It stores all the information that must be shown to the user.
- There are different HTML elements for different types of information and behaviour.
- The information is stored in a tree-like structure (nodes that contain nodes inside) called DOM (Document Object Model).
- It gives the document some semantic structure (pe. this is a title, this is a section, this is a form) which is helpful for computers to understand websites content.
- It must not contain information related to how it should be displayed (that information belongs to the CSS), so no color information, font size, position, etc.

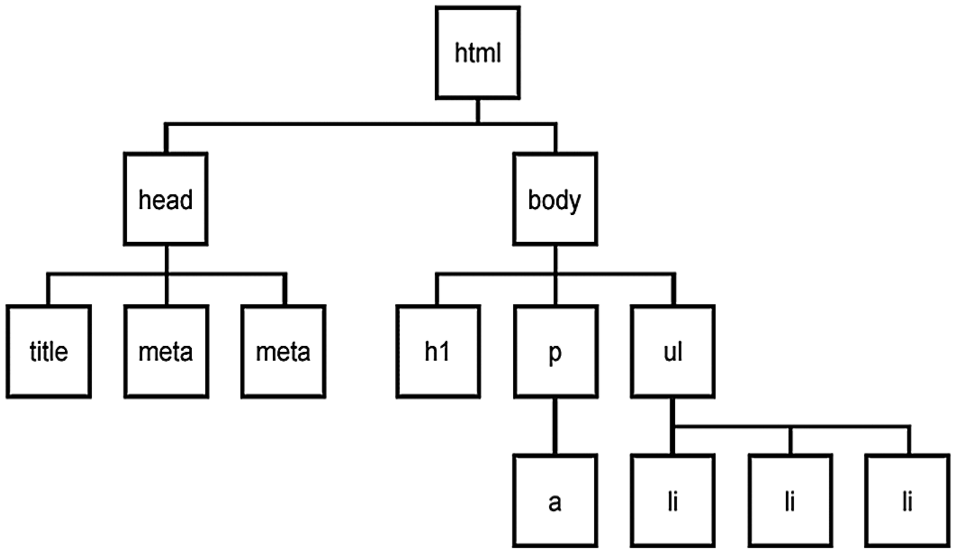
HTML: Syntax example

Tag name
attributes
comment
text tag
self-closing tag

```html
<div id="main">
    <!-- this is a comment -->
    This is text without a tag.
    <button class="mini">press me</button>
    <img src="me.png" />
</div>
```

DOM is a tree

Every node can only have one parent, and every node can have several children, so the structure looks like a tree.



HTML: main tags

Although there are lots of tags in the HTML specification, 99% of the webs use a subset of HTML tags with less that 10 tags, the most important are:

- <div>: a container, usually represents a rectangular area with information inside.
- <img/>: an image
- <a>: a clickable link to go to another URL
- <p>: a text paragraph
- <h1>: a title (h2,h3,h4 are titles of less importance)
- <input>: a widget to let the user introduce information
- <style> and <link>: to insert CSS rules
- <script>: to execute Javascript
- <span>: a null tag (doesn't do anything), good for tagging info

HTML: other tags

There are some tags that could be useful sometimes:

- <button> to create a button
- <audio>: for playing audio
- <video>: to play video
- <canvas>: to draw graphics from javascript
- <iframe>: to put another website inside ours

HTML references

HTML Reference: a description of all HTML tags.

The 25 Most used tags: a list with information of the more common tags.

HTML5 Good practices: some tips for starters

CSS

- CSS Stands for Cascading Style Sheets
- Easy to understand
- Well Organized
- Front-end programming language
- saved with a .css extension when use external CSS.

CSS syntax

```
h1 {
  color: yellow;
  text-align: center;
}
```

- <h1> is the selector in CSS.
- Color is the property and yellow is the value.
- Text-align is the property and center is the value

Ways to insert CSS

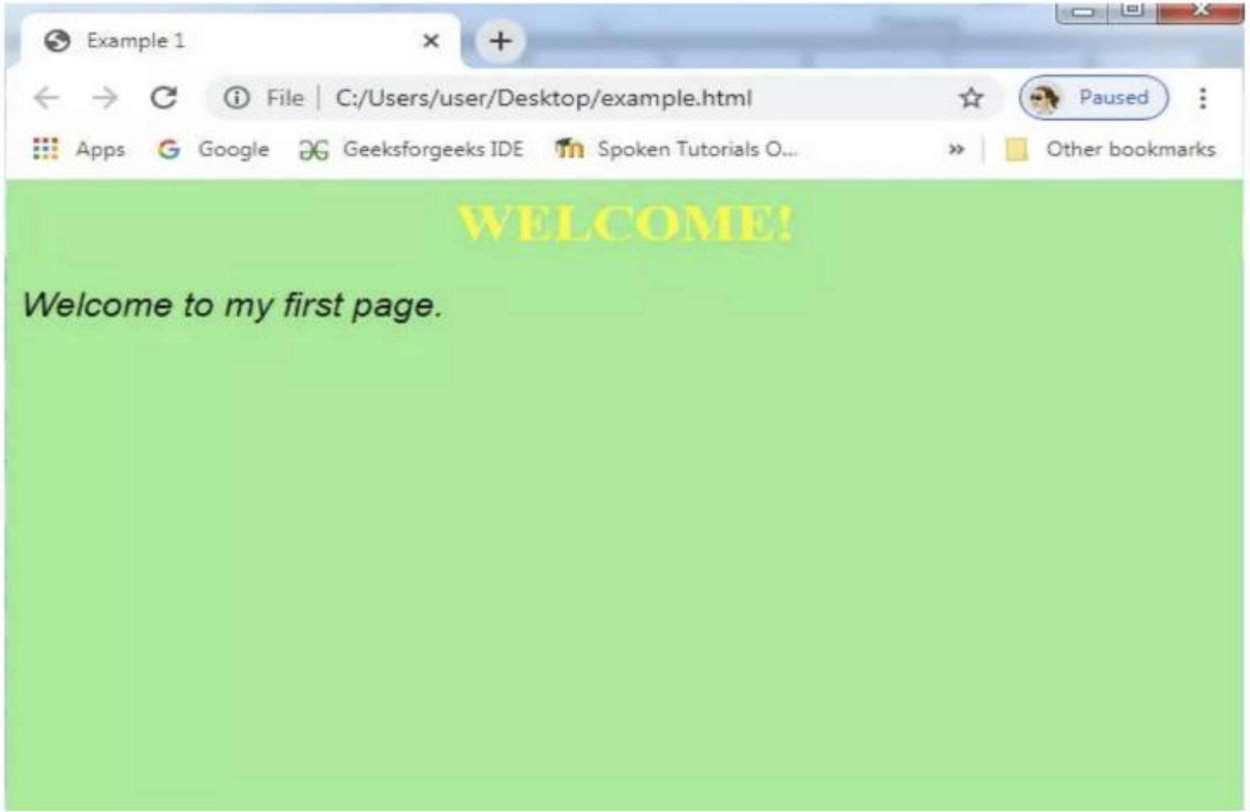
- There are 3 ways to insert CSS:
  1. ***Internal:*** The internal style is defined inside the <style> element, inside the head section.
  2. ***External:*** Can be written in any text editor, and must be saved with a .css extension. The external .css file should not contain any HTML tags.
  3. ***Inline:*** add the style attribute to the element. The style attribute can contain any CSS property.

Internal CSS

```
<!DOCTYPE html>
<html>
<head>
<title> Example 1 </title>
<style>
body {
  background-color: lightgreen;
}

h1 {
  color: yellow;
  text-align: center;
}

p {
  font-family: Arial;
  font-size: 20px;
  font-style:italic;
}
</style>
</head>
<body>
<h1> WELCOME! </h1>
<p>Welcome to my first page.</p>
</body>
</html>
```



Inline CSS

```
<!DOCTYPE html>

<html>
<body style="background-color: lightgreen;">
<h1 style="color:yellow;text-align:center;">This is a
heading</h1>
<p style="font-family: Arial; font-size: 20px; font-
style:italic; ">This is a paragraph.</p>

</body>
</html>
```

External CSS

In Example.html

```
<!DOCTYPE html>
<html>
<head>
<title>Example 1</title>
<link rel="stylesheet" type="text/css"
href="stylesheet1.css">

</head>
<body>

<h1> WELCOME! </h1>
<p>Welcome to my first page.</p>

</body>
</html>
```

In stylesheet.css

```
body {
  background-color: lightgreen;
}

h1 {
  color: yellow;
  text-align: center;
}

p {
  font-family: Arial;
  font-size: 20px;
  font-style:italic;
}
```

CSS fields

Here is a list of the most common CSS fields and an example:

- **color**: #FF0000;   red;   rgba(255,00,100,1.0); //different ways to specify colors
- **background-color**: red;
- **background-image**: url('file.png');
- **font**: 18px 'Tahoma';
- **border**: 2px solid black;
- **border-top**: 2px solid red;
- **border-radius**: 2px; //to remove corners and make them more round
- **margin**: 10px; //distance from the border to the outer elements
- **padding**: 2px; //distance from the border to the inner elements
- width: 100%;   300px;   1.3em; //many different ways to specify distances
- height: 200px;
- text-align: center;
- box-shadow: 3px 3px 5px black;
- cursor: pointer;
- display: inline-block;
- overflow: hidden;

CSS Selectors

There are several selectors we can use to narrow our rules to very specific tags of our website.

The main selectors are:

- **tag name**: just the name of the tag
  - `p { ... }  //affects to all <p> tags`
- **dot (.)**: affects to tags with that class
  - `p.highlight { ... } //affects all <p> tags with class="highlight"`
- **sharp character (#)**: specifies tags with that id
  - `p#intro { ... } //affects to the <p> tag with the id="intro"`
- **two dots (:)**: behaviour states (mouse on top)
  - `p:hover { ... } //affects to <p> tags with the mouse over`
- **brackets ([attr='value'])**: tags with the attribute attr with the value 'value'
  - `input[type="text"] {...} // affects to the input tags of the type text`

CSS selectors

1. Id selector:

- The id of an element is unique within a page.
- The id selector is used to select one unique element.
- Write a hash (#) character, before the id of the element.

2. Class selector:

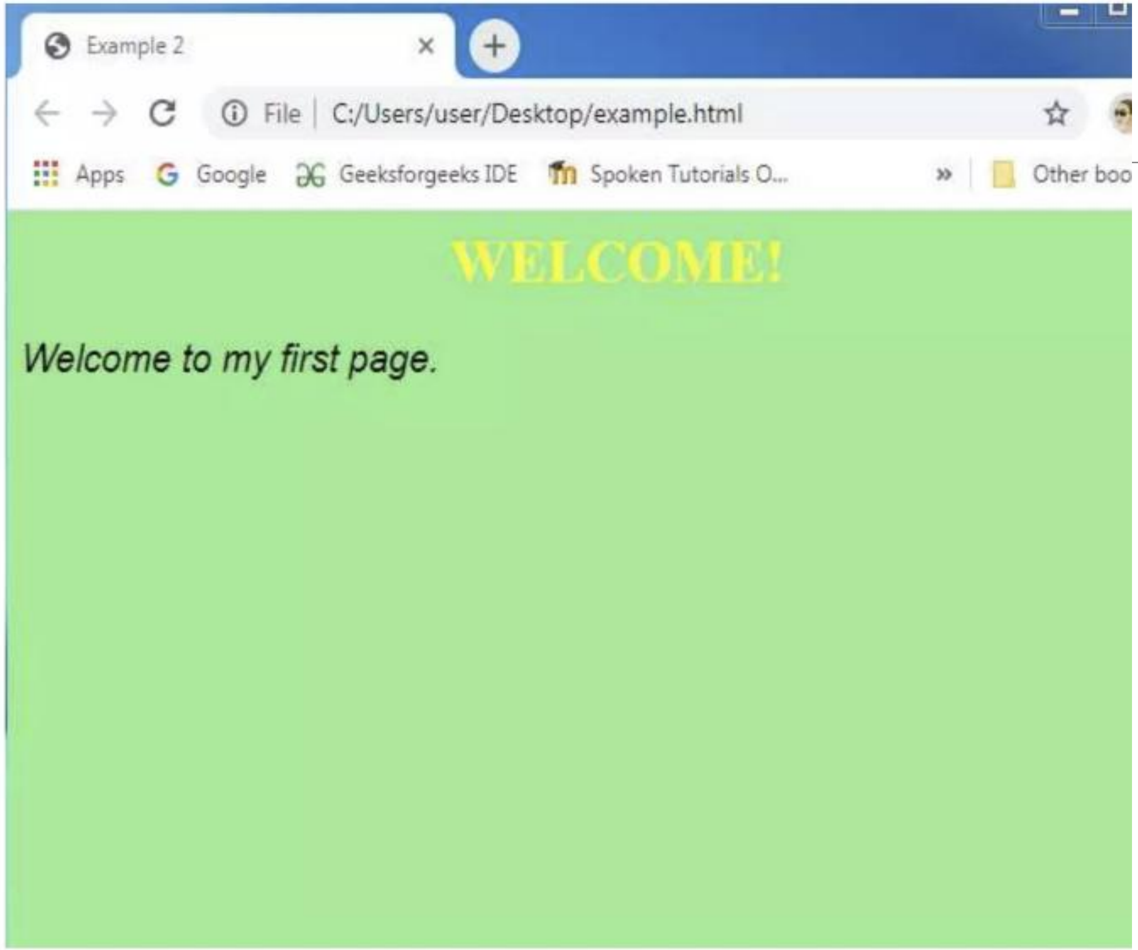
- The class selector selects HTML elements with a specific class attribute.
- To select elements with a specific class, write a dot (.) character, before the class name.

Example

```
<!DOCTYPE html>
<html>
<head>
<title> Example 2</title>
<style>
body {
  background-color: lightgreen;
}

#head1 {
  color: yellow;
  text-align: center;
}

.para1{
  font-family: Arial;
  font-size: 20px;
  font-style:italic;
}
</style>
</head>
<body>
<h1 id="head1"> WELCOME! </h1>
<p class="para1">Welcome to my first page.</p>
</body>
</html>
```



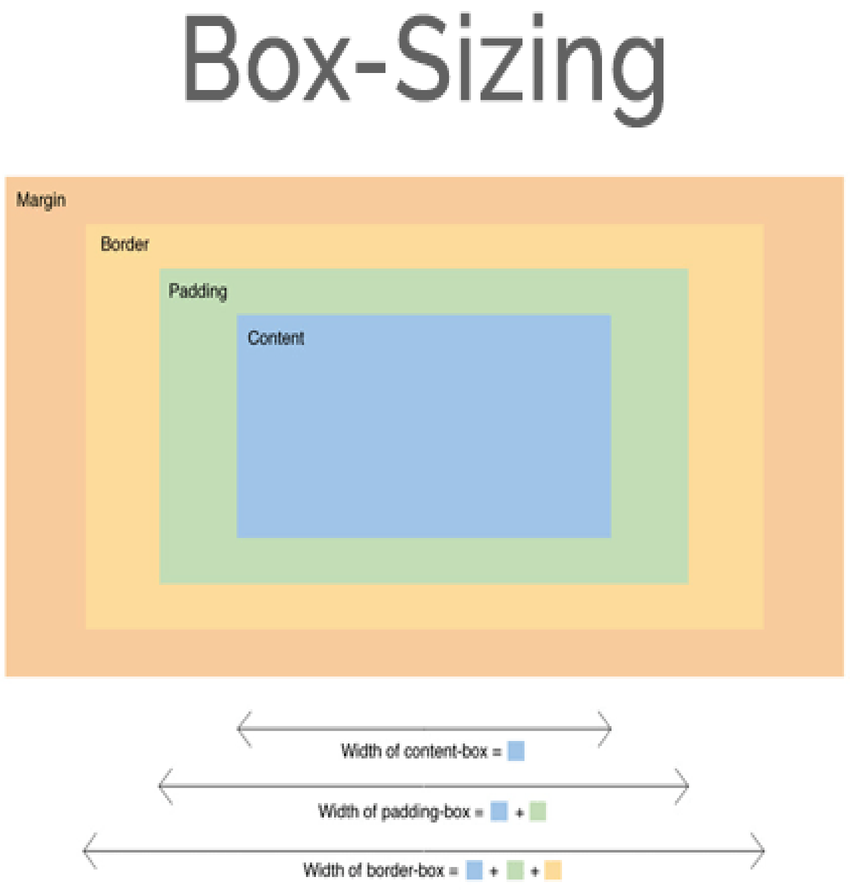
Box Model

It is important to note that by default any width and height specified to an element will not take into account its margin, so a div with width 100px and margin 10px will measure 120px on the screen, not 100px.

This could be a problem breaking your layout.

You can change this behaviour changing the box model of the element so the width uses the outmost border:

```
div { box-sizing: border; }
```

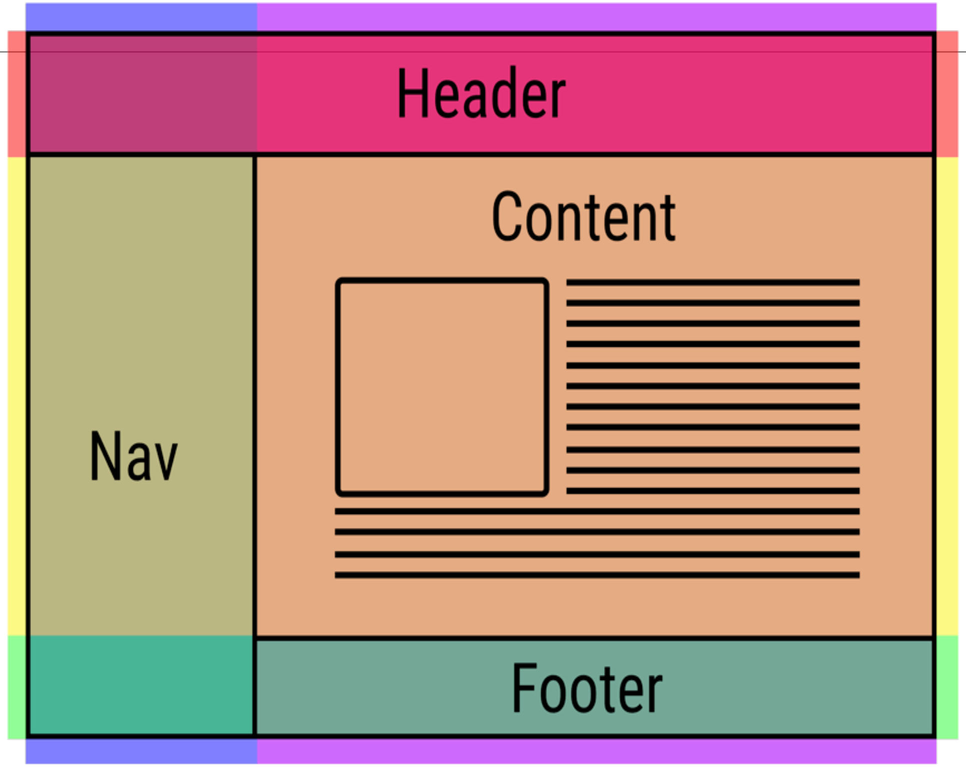


Layout

One of the hardest parts of CSS is constructing the layout of your website (the structure inside the window) .

By default HTML tends to put everything in one column, which is not ideal.

There has been many proposals in CSS to address this issue (tables, fixed divs, flex, grid, …).



JavaScript

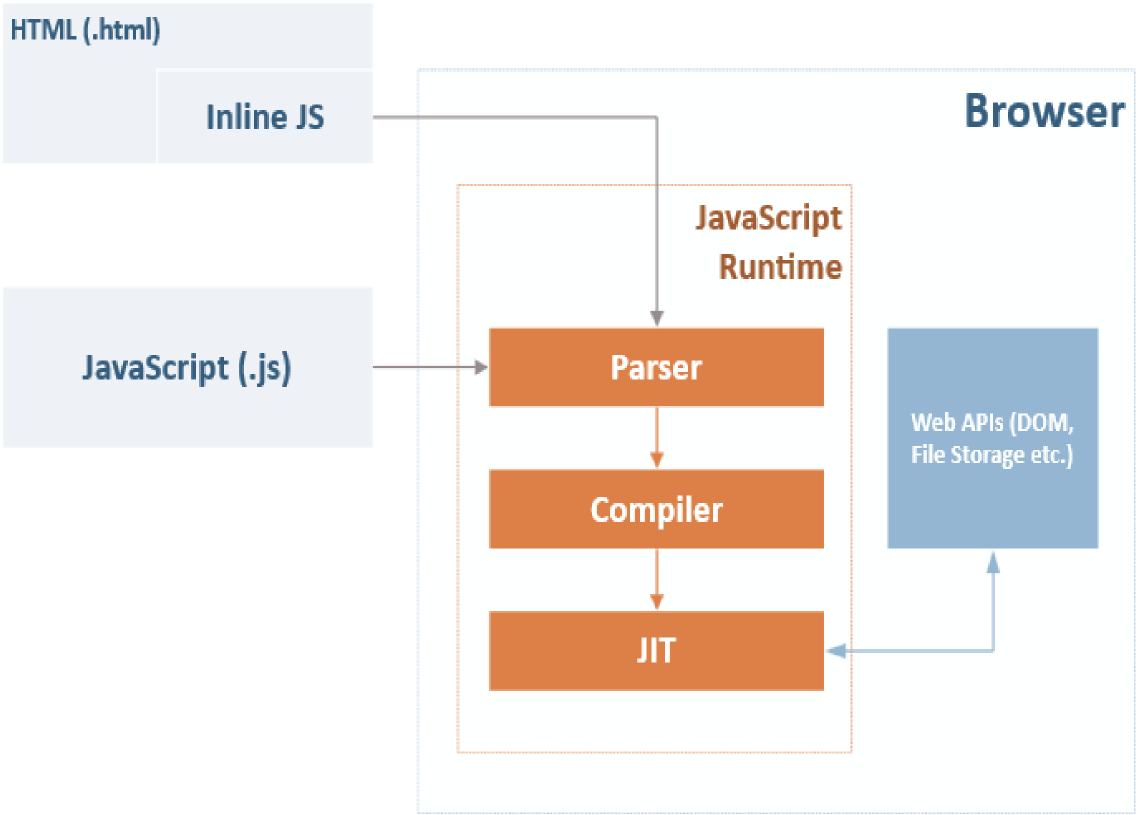
- Is a programming language
- Is used for creating websites
- Easy to learn.
- Standalone language
- Used to make dynamic webpages
- Add special effects on webpages like rollover, roll out and many types of graphics.
- saved with a .js extension.

Javascript Interactivity

Once the web was already being used they realize people wanted to interact with the websites in a more meaningful way.

So they added an Interpreter to execute a script language that could modify the content of the web dynamically.

Brendan Eich was tasked to develop it in one week and it has become one of the most important languages.



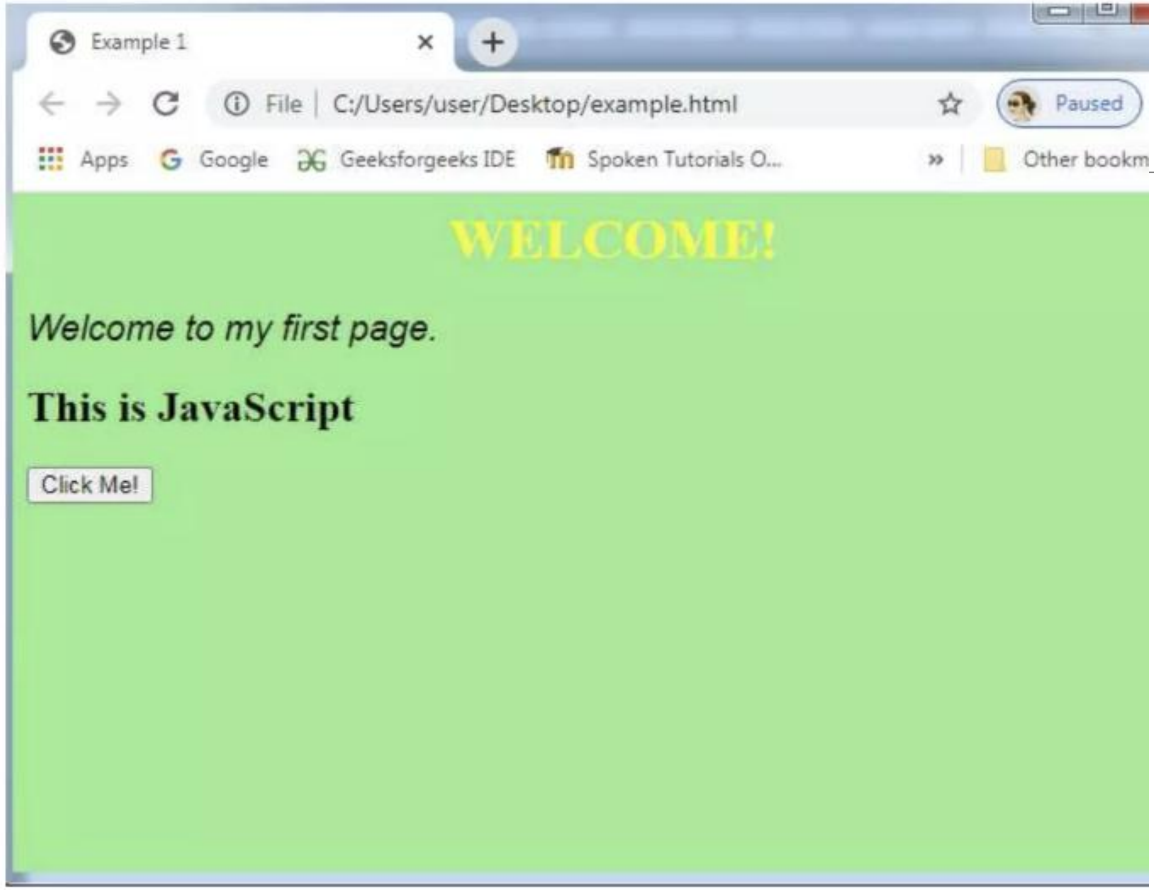
Inline JavaScript

```
<!DOCTYPE html>
<html>
<head>
<title> Example 1</title>
<style>
body {
  background-color: lightgreen;
}

#head1{
  color: yellow;
  text-align: center;
}

.para1 {
  font-family: Arial;
  font-size: 20px;
  font-style:italic;
}
</style>
</head>
<body>
<h1 id="head1"> WELCOME! </h1>
<p class="para1">Welcome to my first page.</p>
<h2 id="head2"></h2>

<button type="button" onclick='document.getElementById("head2").innerHTML = "This is JavaScript!"'>Click Me!</button>
</body>
</html>
```



Internal JavaScript

```
<!DOCTYPE html>
<html>
<head>
<title> Example 1</title>
<style>
body {
  background-color: lightgreen;
}

#head1{
  color: yellow;
  text-align: center;
}

.para1 {
  font-family: Arial;
  font-size: 20px;
  font-style:italic;
}
</style>
<script>
function clickme(){
document.getElementById("head2").innerHTML = "This is JavaScript";
}
</script>
</head>
<body>
<h1 id="head1"> WELCOME! </h1>
<p class="para1">Welcome to my first page.</p>
<h2 id="head2"> </h2>

<button type="button" onclick="clickme()">Click Me!</button>

</body>
</html>
```

External JavaScript

```
<!DOCTYPE html>
<html>
<head>
<title> Example 1</title>
<script  type="text/javascript" src="exjse.js"></script>

<style>
body {
  background-color: lightgreen;
}

#head1{
  color: yellow;
  text-align: center;
}

.para1 {
  font-family: Arial;
  font-size: 20px;
  font-style:italic;
}
</style>

</head>
<body>
<h1 id="head1"> WELCOME! </h1>
<p class="para1">Welcome to my first page.</p>
<h2 id="head2"> </h2>

<button type="button" onclick="clickme()">Click Me!</button>

</body>
</html>
```

Add this code in a new file and name as exjse.js

```
function clickme(){
document.getElementById("head2").innerHTML
= "This is JavaScript";
}
```

Example of a website

HTML in index.html

```
<link href="style.css" rel="stylesheet"/>
<h1>Welcome</h1>
<p>
     <button>Click me</button>
</p>
<script src="code.js"/>
```

CSS in style.css

```
h1 { color: #333; }
button {
     border: 2px solid #AAA;
     background-color: #555;
}
```

Javascript in code.js

```
//fetch the button from the DOM
var button = document.querySelector("button");

//attach and event when the user clicks it
button.addEventListener("click", myfunction);

//create the function that will be called when the button is pressed
function myfunction()
{
          //this shows a popup window
          alert("button clicked!");
}
```